

Real-Time Chat Application

EchoVerse

README

Pranjal Prajapati

Contents

1	Project Overview	1
2	Setting up the Application	2
3	Running the Application Locally	3
4	Future Considerations	3

1 Project Overview

- This project encompasses the task of developing a real-time chat application to facilitate communication between its users.
- It has been built using the following frameworks:
 - **Frontend:** React.js
 - **Backend:** Node.js with Express.js
 - **Real-Time Communication:** Socket.io
 - **Database:** MongoDB
- The application supports the following features:
 - **User Registration and Authentication:** Users can register and create their profiles with a unique username and password. The passwords are first hashed and then stored in the database to prevent them from being compromised on data breach.
 - **Real-Time Messaging:** Application users can send and receive messages in real-time. The messages support both text and emojis.

- **Message History:** Chat history for a particular EchoRoom is maintained and is retrieved whenever the user joins the EchoRoom. This allows the users to view past messages when joining an EchoRoom.
- **User Profile Management:** Users can update their profiles with the application; currently with the option for changing passwords after registration.
- **Complete Separation of EchoRooms :** Each EchoRoom upon creation gets assigned a unique identifier (visible to each member of the group), and users can join the room only upon entering the ID. Each EchoRoom only shows the messages which are sent by members of that room.
- **Encryption of Messages :** All the messages are encrypted before sending to server and decrypted only on the receiver device(s) which are a part of the corresponding EchoRoom.
- **Flexibility on EchoRoom Names:** While usernames for each profile in the application have to be unique for security reasons, users are given the freedom of choosing the name of their choice for their EchoRooms.
- **Status of Other EchoRoom Members:** While inside an EchoRoom, members can view the usernames of all other members of that room which are online at the moment (not available on phone display).

2 Setting up the Application

- To set up the application for running on your local machine, you will require the following to be installed in your system:
 - Node and Node Package Manager (npm)
- Ensure that you have an atlas MongoDB database to which have access permissions to.
- Perform the following steps:
 1. Clone the git repository for this project titled EchoVerse-A-Real-Time-Chat-Application into a new directory (named for example, EchoVerse).
 2. Navigate to the chat-application directory of the repo.
 3. Create a .env file inside the src directory, by copying the .env.example file and replacing the variable values with those corresponding to your own.
 4. Install the front-end dependencies by using the command: npm install.
 5. Now, navigate to the back-end directory of the repo.
 6. Create a .env file in the directory itself, by copying the .env.example file and replacing the variable values with your own.
 7. Install the back-end dependencies by using the command: npm install.

3 Running the Application Locally

- To host the application on your local machine, you will require the following to be installed in your system:
 - Node and Node Package Manager (npm)
 - MongoDB
- Following are the steps for hosting the application on your local machine:
 1. Start a new terminal window and cd into the EchoVerse directory.
 2. Now, cd into the directory back-end and enter the command npm start. This will start up the http server at http://localhost:3001 (default URI for the server).
 3. Keeping this terminal window open, start a new window and cd into the chat-application directory of the repo. Enter the command npm start to start up the application at http://localhost:3000 (default URI for the application)
 4. You should be able to view the login page of the application at the mentioned URL. Congratulations! You have successfully hosted the application on your local machine.

4 Future Considerations

- Implement an authentication token system such as jwt (JSON Web Token) for secure authentication to the application.
- Feature to enable EchoRoom members to send files to one another in a secure and user-friendly manner.
- Feature to enable users to upload an image of their choice as a profile picture. A mechanism would be needed to ensure the file size remains within limit and to monitor the input for any malware.